# PRÁCTICA N.° 09: JAVA SWING BÁSICO: INTRODUCCIÓN Y CONTROLES

| Código | Apellidos y Nombres | Fecha |
|---|---|---|
| 2025002762 | Rodríguez Fádel, Gian Piero Khalil | 11/11/2026 |

## 1. ACTIVIDADES

La actividad de la guía consiste en implementar y demostrar los administradores de esquemas FlowLayout, BorderLayout y GridLayout. En el proyecto se construyó un JFrame principal con tres botones; al presionar cada uno, se abre una ventana independiente. Se comparó visualmente la distribución que produce cada administrador.

### 1.1 Administradores de esquemas y funcionamiento

| Administrador | Funcionamiento | Aplicación observada |
|---|---|---|
| FlowLayout | Coloca componentes en filas de izquierda a derecha y salta de línea cuando ya no hay espacio. | Seis botones con ajuste automático en dos filas. |
| BorderLayout | Divide un contenedor en NORTH, SOUTH, EAST, WEST y CENTER. | Cinco botones ocupan las cinco regiones. |
| GridLayout | Divide el área en celdas iguales de una cuadrícula. | Cinco botones en una matriz de 2 filas y 3 columnas. |

El programa principal utiliza un JPanel con GridLayout(1, 3) para alinear los accesos. La posición de ese JPanel se establece con BorderLayout.NORTH en el JFrame. La apertura de las otras ventanas se realiza con ActionListener en forma de expresiones lambda; DISPOSE_ON_CLOSE permite cerrarlas sin finalizar toda la aplicación.

### 1.2 Evidencias obtenidas en la actividad




*Figura 1. Ventana principal con los accesos a los tres administradores (captura original del estudiante).*

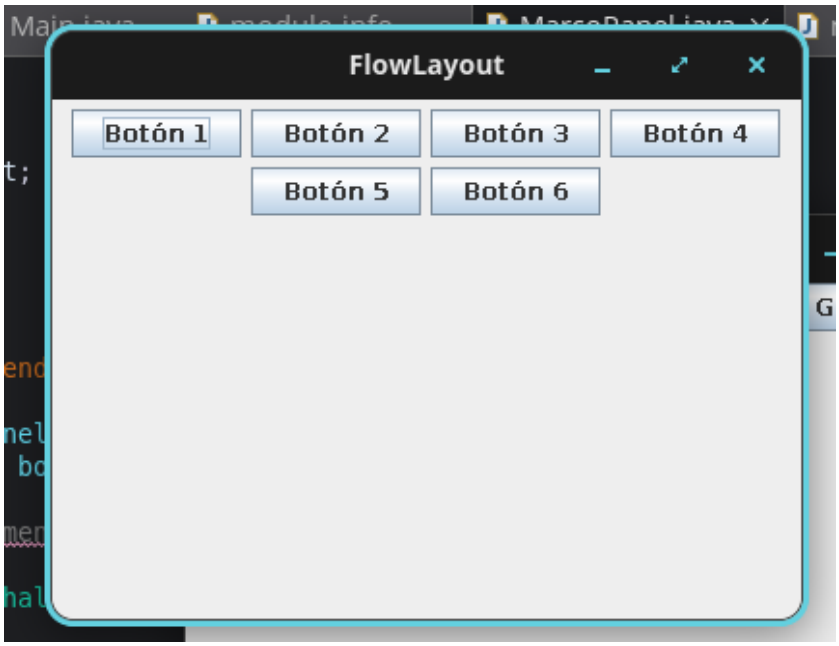



*Figura 2. FlowLayout: seis botones distribuidos horizontalmente y reacomodados en otra fila (captura original).*

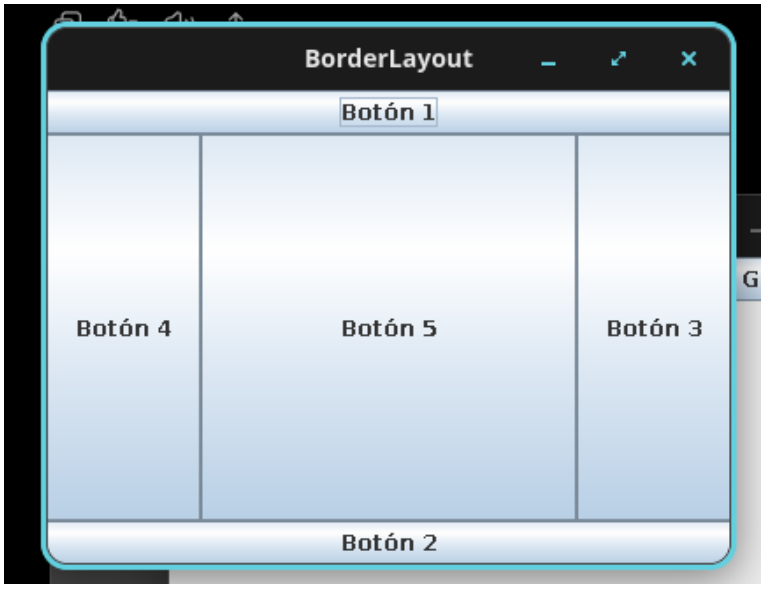



*Figura 3. BorderLayout: botones en las cinco regiones del contenedor (captura original).*

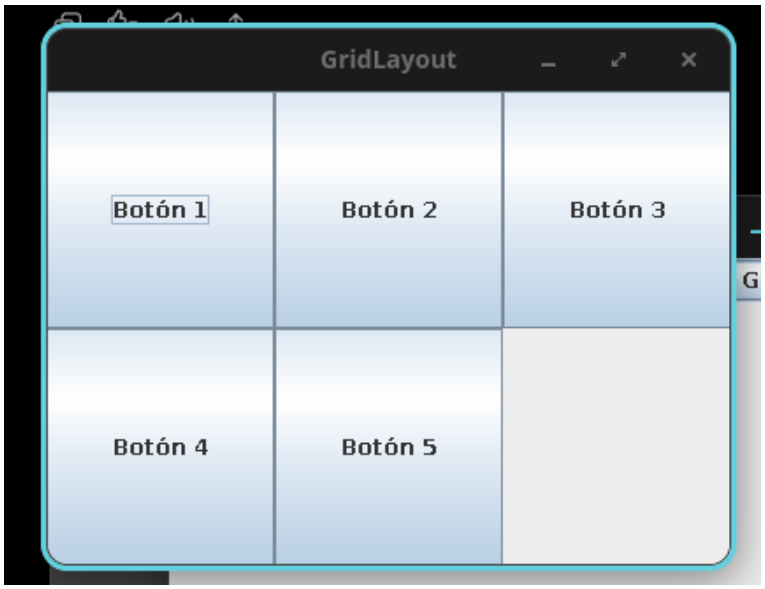



*Figura 4. GridLayout: cinco botones repartidos en celdas uniformes de una cuadrícula 2 × 3 (captura original).*

Nota de trazabilidad: las cuatro capturas anteriores provienen de la ejecución facilitada por el estudiante y muestran el estado de la actividad antes de una pequeña corrección tipográfica del texto del botón BorderLayout en el repositorio. No se alteraron las capturas.

### 1.3 Archivos originales de la actividad y su explicación

### Archivo: actividad_Lab9/src/actividad_Lab9/App.java

Es el punto de entrada principal de la actividad. Crea la ventana MarcoPanel, define que el proceso termine al cerrar el JFrame principal, establece un tamaño inicial de 450 × 200 píxeles y la hace visible. La herencia desde JFrame en App no es necesaria para main, pero se conserva por fidelidad al ejemplo.

```
package actividad_Lab9;

import javax.swing.JFrame;

public class App extends JFrame {

   public static void main(String[] args) {
      MarcoPanel marcoPanel = new MarcoPanel();

      marcoPanel.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      marcoPanel.setSize(450, 200);
      marcoPanel.setVisible(true);
   }
} // fin de la clase App
```

El código anterior se presenta íntegro, tal como se incluye en la versión entregable del repositorio. La estructura de paquetes y la carpeta src deben mantenerse al importarlo en Eclipse.

### Archivo: actividad_Lab9/src/actividad_Lab9/MarcoPanel.java

Esta clase declara el JPanel de la barra de botones y un arreglo de JButton. En su constructor se inicializa GridLayout(1, 3), se crean los accesos, se agregan ActionListener con lambdas y se abre un JFrame por cada botón. FlowLayout usa seis botones, BorderLayout asigna cinco a posiciones definidas y GridLayout construye la cuadrícula 2 × 3. La clase también muestra el nombre del participante en el título principal.

```
package actividad_Lab9;
```

```
import java.awt.GridLayout;
import java.awt.BorderLayout;
import java.awt.FlowLayout;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JButton;

public class MarcoPanel extends JFrame {

   private final JPanel panelBotones; // panel que contiene los botones
   private final JButton[] botones;

   // constructor sin argumentos
   public MarcoPanel() {
      super("Gian Piero Khalil Rodríguez Fádel");

      botones = new JButton[3];
      panelBotones = new JPanel();
      panelBotones.setLayout(new GridLayout(1, botones.length));

      botones[0] = new JButton("FlowLayout");
      botones[1] = new JButton("BorderLayout");
      botones[2] = new JButton("GridLayout");

      panelBotones.add(botones[0]);
      panelBotones.add(botones[1]);
      panelBotones.add(botones[2]);

      botones[0].addActionListener(e ->
      {
            JFrame NuevaVentana = new JFrame("FlowLayout");

            NuevaVentana.setLayout(new FlowLayout());

            NuevaVentana.add(new JButton("Botón 1"));
            NuevaVentana.add(new JButton("Botón 2"));
            NuevaVentana.add(new JButton("Botón 3"));
            NuevaVentana.add(new JButton("Botón 4"));
            NuevaVentana.add(new JButton("Botón 5"));
            NuevaVentana.add(new JButton("Botón 6"));

            NuevaVentana.setSize(400,300);
            NuevaVentana.setLocationRelativeTo(null);
            NuevaVentana.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
            NuevaVentana.setVisible(true);
      });

      botones[1].addActionListener(e ->
      {
            JFrame NuevaVentana = new JFrame("BorderLayout");

            NuevaVentana.setLayout(new BorderLayout());

            NuevaVentana.add(new JButton("Botón 1"), BorderLayout.NORTH);
            NuevaVentana.add(new JButton("Botón 2"), BorderLayout.SOUTH);
            NuevaVentana.add(new JButton("Botón 3"), BorderLayout.EAST);
            NuevaVentana.add(new JButton("Botón 4"), BorderLayout.WEST);
            NuevaVentana.add(new JButton("Botón 5"), BorderLayout.CENTER);

            NuevaVentana.setSize(400,300);
            NuevaVentana.setLocationRelativeTo(null);
            NuevaVentana.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
            NuevaVentana.setVisible(true);
      });

      botones[2].addActionListener(e ->
      {
            JFrame NuevaVentana = new JFrame("GridLayout");

            NuevaVentana.setLayout(new GridLayout(2,3));

            NuevaVentana.add(new JButton("Botón 1"));
```

```
            NuevaVentana.add(new JButton("Botón 2"));
            NuevaVentana.add(new JButton("Botón 3"));
            NuevaVentana.add(new JButton("Botón 4"));
            NuevaVentana.add(new JButton("Botón 5"));

            NuevaVentana.setSize(400,300);
            NuevaVentana.setLocationRelativeTo(null);
            NuevaVentana.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
            NuevaVentana.setVisible(true);
        });


        add(panelBotones, BorderLayout.NORTH); // agrega el panel a JFrame
    }
} // fin de la clase MarcoPanel
```

El código anterior se presenta íntegro, tal como se incluye en la versión entregable del repositorio. La estructura de paquetes y la carpeta src deben mantenerse al importarlo en Eclipse.

### Archivo: actividad_Lab9/src/module-info.java

Declara el módulo actividad_Lab9 y la dependencia requires java.desktop, necesaria para los componentes AWT y Swing. El proyecto Eclipse existente utiliza este descriptor de módulos.

```
/**
 *
 */
/**
 *
 */
module actividad_Lab9 {
            requires java.desktop;
}
```

El código anterior se presenta íntegro, tal como se incluye en la versión entregable del repositorio. La estructura de paquetes y la carpeta src deben mantenerse al importarlo en Eclipse.

# 2. EJERCICIOS PROPUESTOS

## 2.1 Ejercicio 1: creación visual de interfaces Swing

El ejercicio solicita investigar un diseñador visual que permita arrastrar y soltar componentes y generar parte del código Java automáticamente. Se eligió Eclipse WindowBuilder, específicamente Swing Designer, porque se integra al entorno Eclipse utilizado para el proyecto y ofrece una vista visual (Design) y una vista de código (Source). WindowBuilder permite editar propiedades, asignar administradores de esquemas y añadir manejadores de eventos.

### 2.1.1 Procedimiento de diseño con arrastrar y soltar

**Paso 1.** Comprobar que Eclipse dispone del complemento WindowBuilder desde Help > Eclipse Marketplace o mediante su sitio de actualización oficial. Si no está instalado, incorporarlo al IDE.

**Paso 2.** Crear una clase Swing de tipo JFrame o JPanel usando el asistente de WindowBuilder o abrir una clase Swing compatible con el editor WindowBuilder. En el editor seleccionar la pestaña Design.

**Paso 3.** Elegir el administrador de esquemas desde el área de estructura/propiedades. Arrastrar controles de la Palette (JLabel, JTextField, JButton, JComboBox, etc.) y colocarlos en el contenedor. Ajustar propiedades como text y variable.

**Paso 4.** Añadir acciones mediante los manejadores de eventos. Alternar a Source para examinar el código generado y ejecutar la clase como Java Application. Se debe comprobar el comportamiento al cambiar el tamaño de la ventana.

Las dos imágenes siguientes ilustran cómo se verían las fases de inserción de un botón y configuración de propiedades. Son recreaciones académicas hechas para el informe, no capturas de una ejecución verificada de Eclipse/WindowBuilder. Se identifican expresamente como simulaciones para no confundirlas con las capturas reales del proyecto.

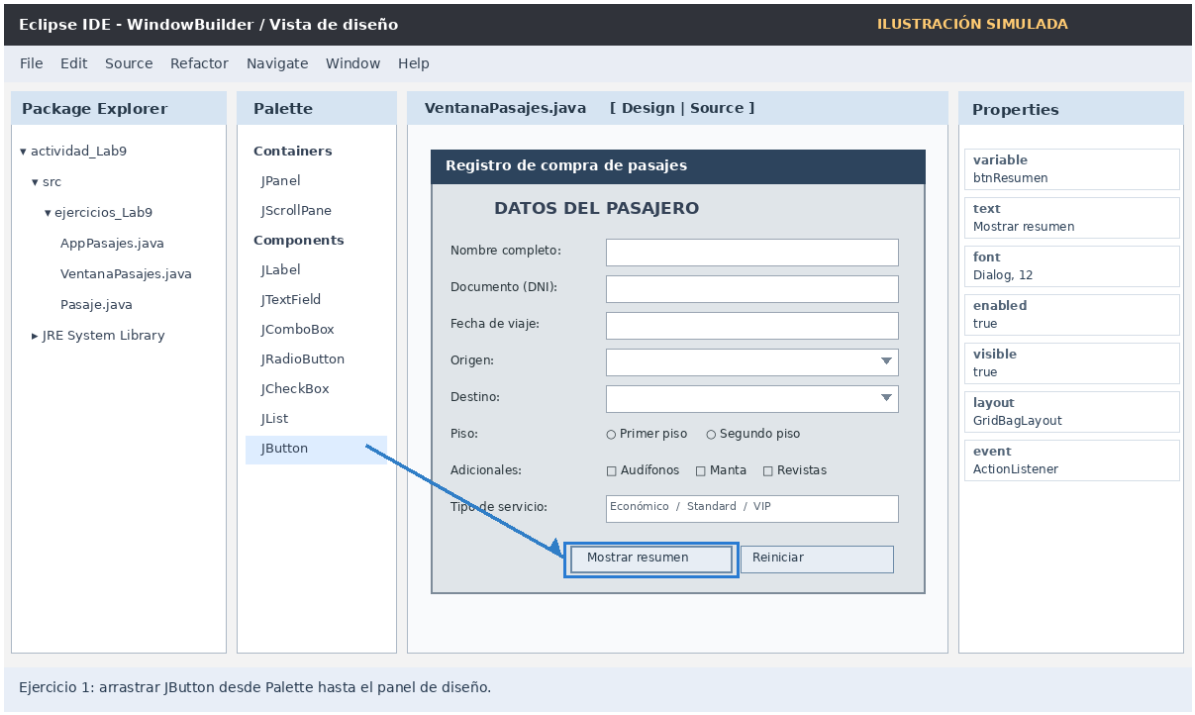



*Figura 5. SIMULACIÓN DIDÁCTICA: arrastre de un JButton desde la paleta de WindowBuilder.*

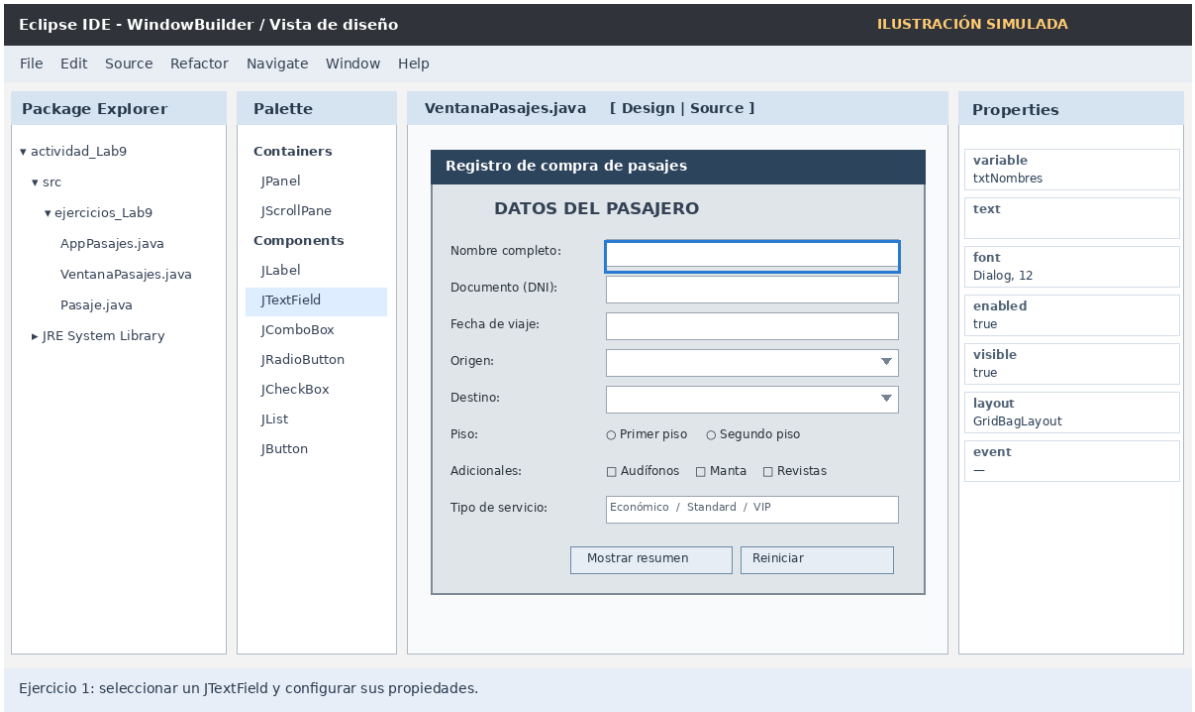



*Figura 6. SIMULACIÓN DIDÁCTICA: selección de un JTextField y revisión de propiedades.*

### 2.1.2 Ventajas y desventajas: código frente a diseñador visual

| Criterio | Creación desde código | Diseñador visual |
|---|---|---|
| Control del diseño | Permite ajustar al detalle los contenedores, lógica y eventos; puede requerir más tiempo. | Facilita colocar, alinear y dimensionar los controles con menor esfuerzo inicial. |
| Comprensión | Se observa directamente el funcionamiento de JFrame, JPanel y LayoutManager. | El código generado puede ocultar parte de las decisiones de distribución. |
| Mantenimiento | Más flexible para refactorizar, reutilizar y revisar cambios mediante Git. | Puede generar archivos extensos o dificultar modificaciones manuales complejas. |
| Rapidez | Requiere escribir y probar los controles manualmente. | Resulta más rápido para prototipos y formularios habituales. |
| Dependencia | Solo requiere el JDK y bibliotecas utilizadas. | El diseño asistido requiere el complemento en el equipo de desarrollo; la ejecución del código generado no necesariamente lo requiere. |

Se concluye que ambas formas son válidas. Para interfaces grandes, la herramienta visual puede acelerar la organización, mientras que el desarrollo manual facilita practicar los conceptos. En este laboratorio la aplicación funcional del ejercicio 2 se desarrolló por código y las pantallas de WindowBuilder se presentan exclusivamente como ejemplos ilustrativos.

## 2.2 Ejercicio 2: aplicación de compra de pasajes

El ejercicio indica desarrollar una aplicación Swing que simule el ingreso de datos para comprar pasajes de una empresa de transporte terrestre. Una vez ingresada la información, un botón debe mostrar un resumen en un cuadro de diálogo. Se implementó una aplicación de escritorio separada en tres clases para distinguir la interfaz, los datos y el punto de entrada.

### 2.2.1 Controles solicitados e implementación

| Control Swing | Uso en la aplicación |
|---|---|
| JLabel | Identifica cada dato del pasajero: nombres, documento, fecha, origen, destino, piso y servicio. |
| JTextField | Permite escribir nombres completos, un DNI de ocho dígitos y la fecha de viaje. |
| JButton | Mostrar resumen valida y presenta el resultado; Reiniciar formulario elimina lo ingresado. |
| JCheckBox | Audífonos, manta y revistas se pueden seleccionar de manera independiente. |
| JRadioButton + ButtonGroup | Permite escoger solo una opción: primer o segundo piso. |
| JComboBox | Brinda listas desplegables con ciudades de origen y destino. |
| JList | Permite escoger una sola calidad de servicio: |

| | |
|---|---|
| | Económico, Standard o VIP. |
| JOptionPane | Muestra avisos cuando los datos son inválidos y el resumen cuando son correctos. |

La ventana utiliza BorderLayout para separar título, formulario y botones. GridBagLayout alinea etiquetas y campos en filas y columnas; FlowLayout agrupa radio botones, casillas y acciones. El proyecto demuestra así el empleo combinado de componentes y administradores de esquemas. Los registros se mantienen únicamente en memoria; no hay procesamiento de pagos, almacenamiento persistente ni reservas reales.

### 2.2.2 Validación y manejo de eventos

Al pulsar Mostrar resumen, un ActionListener llama a mostrarResumen(). Este método ejecuta construirPasaje(), valida los campos y crea el objeto Pasaje. En caso de error se lanza IllegalArgumentException y se abre un JOptionPane de advertencia con la causa. Si los campos son válidos, Pasaje.generarResumen() reúne los datos y se presentan en un JOptionPane informativo.

Las validaciones implementadas comprueban: nombres no vacíos; DNI compuesto de ocho dígitos; fecha real en formato dd/MM/aaaa, sin fecha anterior a la actual; ciudades diferentes; piso elegido y una categoría de servicio seleccionada. Los adicionales son opcionales. El botón de reinicio deja los campos vacíos, restablece los combos y desmarca los controles.

### 2.2.3 Código completo del ejercicio 2

### Archivo: actividad_Lab9/src/ejercicios_Lab9/AppPasajes.java

El método main es la entrada del segundo ejercicio. SwingUtilities.invokeLater() inicia la creación de los controles en el Event Dispatch Thread, evitando hacer operaciones de interfaz desde un hilo ajeno. Se intenta seleccionar el aspecto Nimbus cuando está instalado; de lo contrario se conserva el predeterminado. Finalmente, se muestra VentanaPasajes.

```
package ejercicios_Lab9;

import javax.swing.SwingUtilities;
import javax.swing.UIManager;

/** Punto de entrada del ejercicio 2 del Laboratorio 09. */
public class AppPasajes {
   public static void main(String[] args) {
      SwingUtilities.invokeLater(() -> {
         try {
            for (UIManager.LookAndFeelInfo estilo : UIManager.getInstalledLookAndFeels()) {
               if ("Nimbus".equals(estilo.getName())) {
                  UIManager.setLookAndFeel(estilo.getClassName());
                  break;
               }
            }
         } catch (Exception ex) {
            System.err.println("Apariencia predeterminada: " + ex.getMessage());
         }
         new VentanaPasajes().setVisible(true);
      });
   }
}
```

El código anterior se presenta íntegro, tal como se incluye en la versión entregable del repositorio. La estructura de paquetes y la carpeta src deben mantenerse al importarlo en Eclipse.

**Archivo: actividad_Lab9/src/ejercicios_Lab9/Pasaje.java**

Clase de datos del pasaje. Sus atributos son private final y se inicializan mediante el constructor. List.copyOf() evita que la lista de servicios adicionales se modifique desde fuera. generarResumen() arma una cadena legible a partir de la información capturada, incluida la fecha formateada mediante DateTimeFormatter.

```
package ejercicios_Lab9;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.List;

/** Datos de una solicitud de pasaje. Es un modelo en memoria, sin persistencia. */
public class Pasaje {
   private final String pasajero;
   private final String documento;
   private final LocalDate fecha;
   private final String origen;
   private final String destino;
   private final String piso;
   private final String servicio;
   private final List<String> adicionales;

   public Pasaje(String pasajero, String documento, LocalDate fecha,
                 String origen, String destino, String piso,
                 String servicio, List<String> adicionales) {
      this.pasajero = pasajero;
      this.documento = documento;
      this.fecha = fecha;
      this.origen = origen;
      this.destino = destino;
      this.piso = piso;
      this.servicio = servicio;
      this.adicionales = List.copyOf(adicionales);
   }

   /** Genera el texto que se muestra en JOptionPane. */
   public String generarResumen() {
      DateTimeFormatter formato = DateTimeFormatter.ofPattern("dd/MM/uuuu");
      String extras = adicionales.isEmpty()
            ? "Ninguno" : String.join(", ", adicionales);

      return "RESUMEN DE COMPRA DE PASAJE\n"
            + "----------------------------------------\n"
            + "Pasajero: " + pasajero + "\n"
            + "Documento: " + documento + "\n"
            + "Fecha de viaje: " + fecha.format(formato) + "\n"
            + "Origen: " + origen + "\n"
            + "Destino: " + destino + "\n"
            + "Ubicación: " + piso + "\n"
            + "Servicio: " + servicio + "\n"
            + "Adicionales: " + extras + "\n"
            + "----------------------------------------\n"
            + "Solicitud ingresada (simulación).";
   }
}
```

El código anterior se presenta íntegro, tal como se incluye en la versión entregable del repositorio. La estructura de paquetes y la carpeta src deben mantenerse al importarlo en Eclipse.

**Archivo: actividad_Lab9/src/ejercicios_Lab9/VentanaPasajes.java**

Clase responsable de toda la interfaz Swing. Define los JTextField, JComboBox, JRadioButton, JCheckBox, JList y JButton. El constructor crea el JFrame, configura layouts y bordes, agrupa las opciones de piso con ButtonGroup y registra eventos para los botones. agregarFila() evita repetir la configuración de GridBagConstraints; mostrarResumen() gestiona el diálogo, construirPasaje()

valida los datos y reiniciarFormulario() restaura el estado inicial. Este archivo contiene la lógica principal del ejercicio.

```
package ejercicios_Lab9;

import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Insets;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.time.format.ResolverStyle;
import java.util.ArrayList;
import java.util.List;

import javax.swing.BorderFactory;
import javax.swing.BoxLayout;
import javax.swing.ButtonGroup;
import javax.swing.JButton;
import javax.swing.JCheckBox;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JList;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JRadioButton;
import javax.swing.JScrollPane;
import javax.swing.JTextField;
import javax.swing.ListSelectionModel;
import javax.swing.SwingConstants;

/** Ejercicio 2: formulario de compra de pasajes de transporte terrestre. */
public class VentanaPasajes extends JFrame {
   private final JTextField txtNombres = new JTextField(24);
   private final JTextField txtDocumento = new JTextField(24);
   private final JTextField txtFecha = new JTextField(24);

   private final JComboBox<String> cmbOrigen = new JComboBox<>(new String[] {
         "Seleccione origen", "Arequipa", "Lima", "Cusco", "Tacna", "Puno", "Moquegua"
   });
   private final JComboBox<String> cmbDestino = new JComboBox<>(new String[] {
         "Seleccione destino", "Arequipa", "Lima", "Cusco", "Tacna", "Puno", "Moquegua"
   });

   private final JRadioButton rbPrimerPiso = new JRadioButton("Primer piso");
   private final JRadioButton rbSegundoPiso = new JRadioButton("Segundo piso");
   private final JCheckBox chkAudifonos = new JCheckBox("Audífonos");
   private final JCheckBox chkManta = new JCheckBox("Manta");
   private final JCheckBox chkRevistas = new JCheckBox("Revistas");

   private final JList<String> listaServicios = new JList<>(new String[] {
         "Económico", "Standard", "VIP"
   });
   private final JButton btnResumen = new JButton("Mostrar resumen");
   private final JButton btnLimpiar = new JButton("Reiniciar formulario");

   public VentanaPasajes() {
      super("Compra de pasajes - Transporte Santa María");
      setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
      setSize(790, 665);
      setMinimumSize(new java.awt.Dimension(680, 590));
      setLocationRelativeTo(null);
      setLayout(new BorderLayout(10, 10));

      JLabel titulo = new JLabel("REGISTRO DE COMPRA DE PASAJES", SwingConstants.CENTER);
      titulo.setFont(titulo.getFont().deriveFont(java.awt.Font.BOLD, 19f));
      titulo.setBorder(BorderFactory.createEmptyBorder(18, 6, 8, 6));
      add(titulo, BorderLayout.NORTH);
```

```
    JPanel formulario = new JPanel(new GridBagLayout());
    formulario.setBorder(BorderFactory.createTitledBorder("Información del pasajero y viaje"));
    GridBagConstraints c = new GridBagConstraints();
    c.insets = new Insets(8, 12, 8, 12);
    c.anchor = GridBagConstraints.WEST;
    c.fill = GridBagConstraints.HORIZONTAL;

    agregarFila(formulario, c, 0, "Nombres y apellidos:", txtNombres);
    agregarFila(formulario, c, 1, "Documento de identidad:", txtDocumento);
    txtFecha.setToolTipText("Formato dd/MM/aaaa; ejemplo: 20/11/2026");
    agregarFila(formulario, c, 2, "Fecha de viaje (dd/MM/aaaa):", txtFecha);
    agregarFila(formulario, c, 3, "Ciudad de origen:", cmbOrigen);
    agregarFila(formulario, c, 4, "Ciudad de destino:", cmbDestino);

    ButtonGroup grupoPiso = new ButtonGroup();
    grupoPiso.add(rbPrimerPiso);
    grupoPiso.add(rbSegundoPiso);
    JPanel panelPiso = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
    panelPiso.add(rbPrimerPiso);
    panelPiso.add(rbSegundoPiso);
    agregarFila(formulario, c, 5, "Ubicación del asiento:", panelPiso);

    JPanel panelExtras = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
    panelExtras.add(chkAudifonos);
    panelExtras.add(chkManta);
    panelExtras.add(chkRevistas);
    agregarFila(formulario, c, 6, "Servicios adicionales:", panelExtras);

    listaServicios.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
    listaServicios.setVisibleRowCount(3);
    JScrollPane scrollServicios = new JScrollPane(listaServicios);
    agregarFila(formulario, c, 7, "Calidad de servicio:", scrollServicios);
    add(formulario, BorderLayout.CENTER);

    JPanel inferior = new JPanel();
    inferior.setLayout(new BoxLayout(inferior, BoxLayout.Y_AXIS));
    JPanel acciones = new JPanel(new FlowLayout(FlowLayout.CENTER, 15, 9));
    acciones.add(btnResumen);
    acciones.add(btnLimpiar);
    inferior.add(acciones);
    JLabel aclaracion = new JLabel("Simulación académica: no realiza pagos ni reservas.", SwingConstants.CENTER);
    aclaracion.setAlignmentX(CENTER_ALIGNMENT);
    inferior.add(aclaracion);
    inferior.setBorder(BorderFactory.createEmptyBorder(0, 10, 12, 10));
    add(inferior, BorderLayout.SOUTH);

    btnResumen.addActionListener(e -> mostrarResumen());
    btnLimpiar.addActionListener(e -> reiniciarFormulario());
}

private void agregarFila(JPanel panel, GridBagConstraints c,
                         int fila, String etiqueta, java.awt.Component componente) {
    c.gridx = 0;
    c.gridy = fila;
    c.weightx = 0;
    panel.add(new JLabel(etiqueta), c);
    c.gridx = 1;
    c.weightx = 1;
    panel.add(componente, c);
}

private void mostrarResumen() {
    try {
        Pasaje pasaje = construirPasaje();
        JOptionPane.showMessageDialog(this, pasaje.generarResumen(),
                "Resumen del pasaje", JOptionPane.INFORMATION_MESSAGE);
    } catch (IllegalArgumentException error) {
        JOptionPane.showMessageDialog(this, error.getMessage(),
                "Datos incompletos o inválidos", JOptionPane.WARNING_MESSAGE);
    }
}
```

```
    private Pasaje construirPasaje() {
        String nombre = txtNombres.getText().trim();
        String documento = txtDocumento.getText().trim();
        if (nombre.isBlank()) {
            throw new IllegalArgumentException("Ingrese los nombres y apellidos del pasajero.");
        }
        if (!documento.matches("[0-9]{8}")) {
            throw new IllegalArgumentException("El documento debe contener ocho dígitos (DNI).");
        }

        LocalDate fecha;
        try {
            DateTimeFormatter formato = DateTimeFormatter.ofPattern("dd/MM/uuuu")
                    .withResolverStyle(ResolverStyle.STRICT);
            fecha = LocalDate.parse(txtFecha.getText().trim(), formato);
        } catch (DateTimeParseException error) {
            throw new IllegalArgumentException("La fecha debe ser válida y usar dd/MM/aaaa.");
        }
        if (fecha.isBefore(LocalDate.now())) {
            throw new IllegalArgumentException("La fecha de viaje no puede ser anterior a hoy.");
        }

        if (cmbOrigen.getSelectedIndex() == 0 || cmbDestino.getSelectedIndex() == 0) {
            throw new IllegalArgumentException("Seleccione las ciudades de origen y destino.");
        }
        String origen = (String) cmbOrigen.getSelectedItem();
        String destino = (String) cmbDestino.getSelectedItem();
        if (origen.equals(destino)) {
            throw new IllegalArgumentException("El origen y el destino deben ser diferentes.");
        }
        if (!rbPrimerPiso.isSelected() && !rbSegundoPiso.isSelected()) {
            throw new IllegalArgumentException("Indique si desea viajar en el primer o segundo piso.");
        }
        if (listaServicios.getSelectedValue() == null) {
            throw new IllegalArgumentException("Seleccione la calidad de servicio.");
        }

        List<String> adicionales = new ArrayList<>();
        if (chkAudifonos.isSelected()) adicionales.add("Audífonos");
        if (chkManta.isSelected()) adicionales.add("Manta");
        if (chkRevistas.isSelected()) adicionales.add("Revistas");

        return new Pasaje(nombre, documento, fecha, origen, destino,
                rbPrimerPiso.isSelected() ? "Primer piso" : "Segundo piso",
                listaServicios.getSelectedValue(), adicionales);
    }

    private void reiniciarFormulario() {
        txtNombres.setText("");
        txtDocumento.setText("");
        txtFecha.setText("");
        cmbOrigen.setSelectedIndex(0);
        cmbDestino.setSelectedIndex(0);
        rbPrimerPiso.setSelected(false);
        rbSegundoPiso.setSelected(false);
        chkAudifonos.setSelected(false);
        chkManta.setSelected(false);
        chkRevistas.setSelected(false);
        listaServicios.clearSelection();
        txtNombres.requestFocusInWindow();
    }
}
```

El código anterior se presenta íntegro, tal como se incluye en la versión entregable del repositorio. La estructura de paquetes y la carpeta src deben mantenerse al importarlo en Eclipse.

### 2.2.4 Evidencias de ejecución funcional

Las capturas siguientes se obtuvieron al ejecutar las clases Java compiladas en un servidor gráfico virtual (Xvfb) con un administrador de ventanas. Se utilizaron datos de prueba y se accionaron

programáticamente los controles, por lo que documentan ejecuciones reales controladas, no una captura del escritorio personal del estudiante. La Figura 6 anterior, por el contrario, sí es una ilustración simulada de WindowBuilder.




*Figura 7. Captura real en entorno de pruebas: formulario inicial, sin datos.*

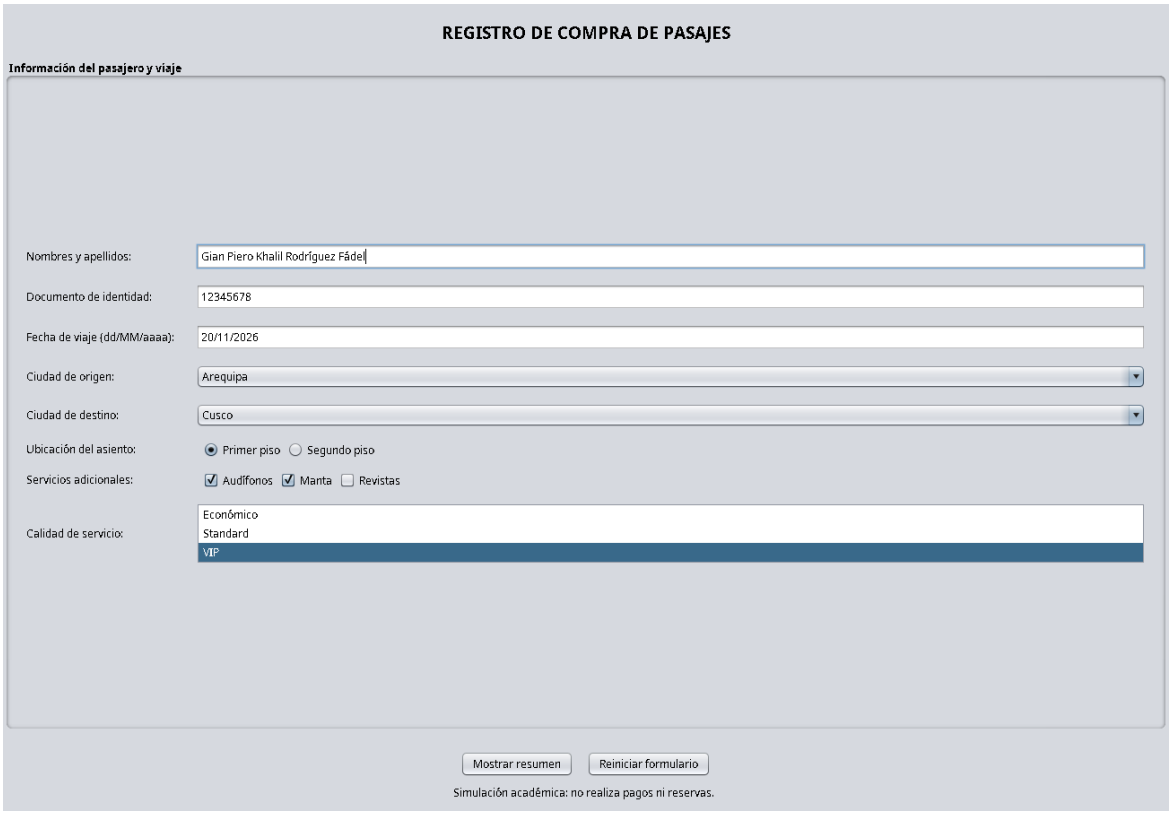



*Figura 8. Captura real en entorno de pruebas: datos de pasajero, ciudades, piso, extras y VIP.*

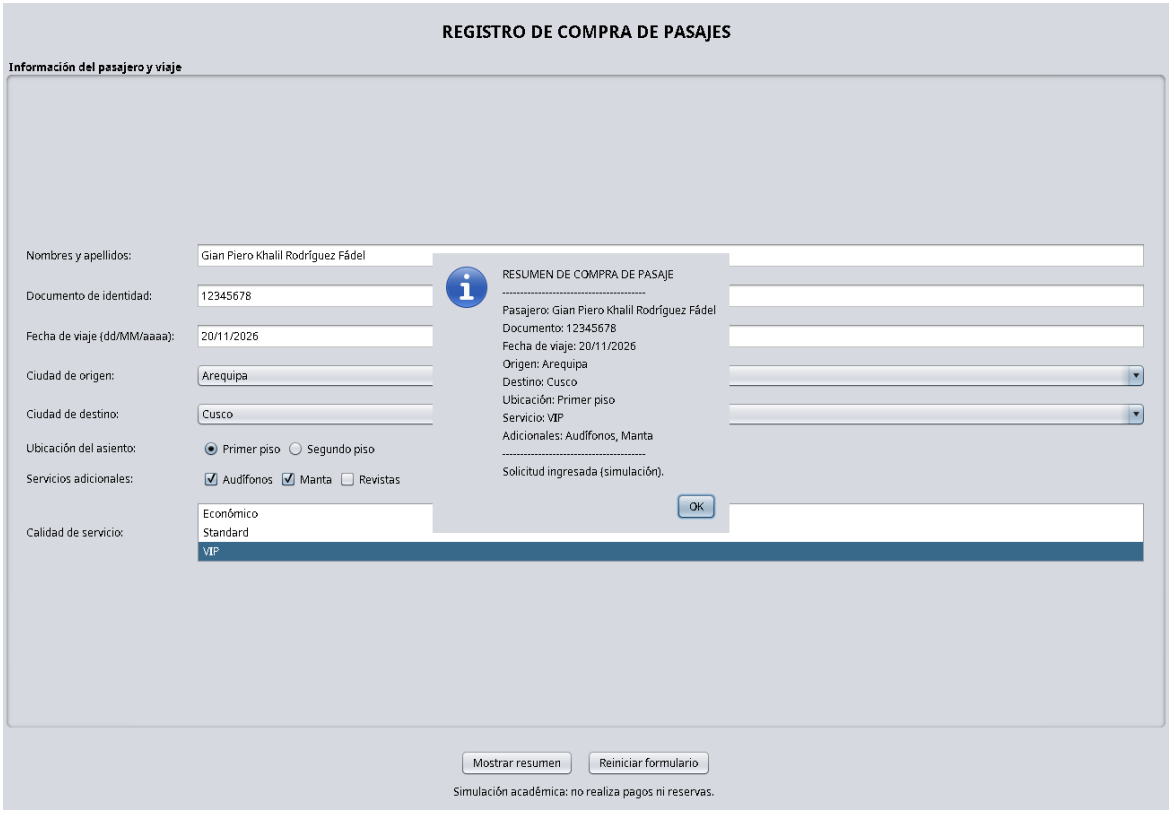



*Figura 9. Captura real en entorno de pruebas: cuadro JOptionPane con el resumen del pasaje.*




*Figura 10. Captura real en entorno de pruebas: formulario limpio después de pulsar Reiniciar.*

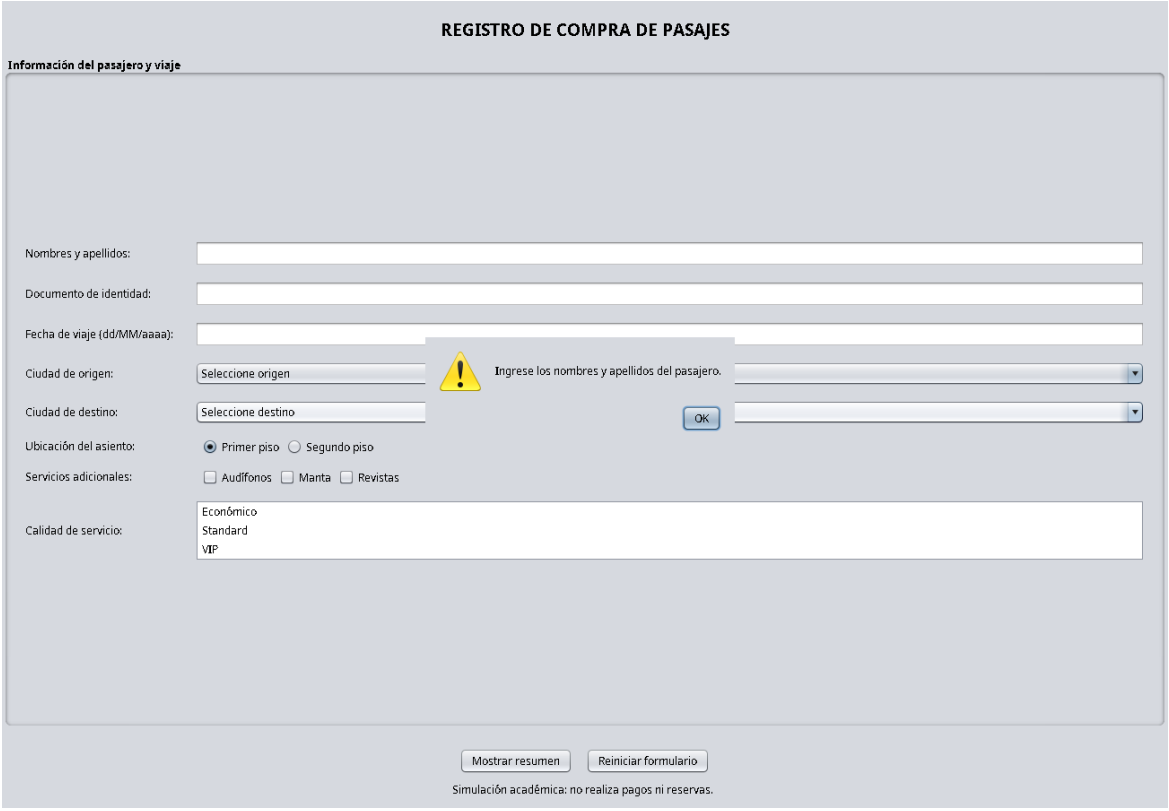



*Figura 11. Captura real en entorno de pruebas: advertencia de validación al intentar enviar el formulario vacío.*

### 2.2.5 Resultados y verificación

| **Prueba** | **Acción realizada** | **Resultado observado** |
|---|---|---|
| Inicio | Ejecutar ejercicios_Lab9.AppPasajes. | Se visualizó el formulario con los controles exigidos. |
| Datos completos | Llenar campos válidos y elegir servicio. | Se obtuvo el resumen con los valores seleccionados. |
| Reinicio | Pulsar Reiniciar formulario. | Los campos y selecciones quedaron vacíos. |
| Campos obligatorios | Pulsar Mostrar resumen con el formulario vacío. | Se mostró una advertencia que indica el campo faltante. |
| Compilación | Compilar las clases con javac de Java 21. | Compilación correcta; advertencia informativa por nombre de módulo con dígitos. |

La validación realizada cubre el flujo principal de interacción. Para comprobar el comportamiento en el equipo de entrega se debe importar el proyecto Eclipse, verificar que la configuración de Java sea compatible y ejecutar las clases principales. La evidencia de WindowBuilder es una ilustración de procedimiento y no demuestra la instalación del complemento en ese equipo.

## 3. CONCLUSIONES

Se reconocieron las diferencias prácticas entre FlowLayout, BorderLayout y GridLayout mediante ventanas independientes y capturas de la actividad. Se investigó cómo un diseñador visual puede

acelerar la creación de GUI y se identificaron sus diferencias con la programación manual. Finalmente, se desarrolló un formulario Swing funcional que utiliza los controles exigidos, responde a los eventos de los botones, comprueba los datos ingresados y muestra el resumen del pasaje. Se distinguieron las capturas originales, las ejecuciones controladas y las ilustraciones simuladas para mantener la trazabilidad del informe.

## 4. BIBLIOGRAFÍA

Universidad Católica de Santa María. (2026). Guía 09: Java Swing básico: introducción y controles. Laboratorio de Lenguajes de Programación III.

Oracle. (s. f.). A visual guide to layout managers. The Java Tutorials. https://docs.oracle.com/javase/tutorial/uiswing/layout/visual.html

Oracle. (s. f.). How to use FlowLayout. The Java Tutorials. https://docs.oracle.com/javase/tutorial/uiswing/layout/flow.html

Eclipse Foundation. (s. f.). WindowBuilder: A Java GUI designer. https://eclipse.dev/windowbuilder/

Eclipse Foundation. (s. f.). WindowBuilder FAQ. https://help.eclipse.org/latest/topic/org.eclipse.wb.doc.user/html/faq.html

Rodríguez Fádel, G. P. K. (2026). Lab9_LP3 [Repositorio de código fuente]. GitHub. https://github.com/Crudo27/Lab9_LP3

# 5. ANEXOS

### Anexo A. Repositorio del Laboratorio 09

El proyecto del Laboratorio 09 de Lenguajes de Programación III se encuentra disponible en el siguiente repositorio público de GitHub. Incluye la implementación de los administradores de esquemas, el ejercicio de compra de pasajes, el código fuente Java y las evidencias documentadas.

**Enlace del repositorio:** https://github.com/Crudo27/Lab9_LP3

Rama principal: main.

Código fuente: actividad_Lab9/src/.

Informe y capturas de evidencia: documentacion/.

Este enlace permite revisar los archivos utilizados para la resolución de las actividades y ejercicios de la guía, así como las evidencias identificadas en el informe.